Model card: GreenShift carbon-aware batch scheduler

Format after Mitchell et al., *Model Cards for Model Reporting* (FAT* 2019). GreenShift is a rule-based decision system, not a trained model: it has no learned weights and no training data, so the "training data" section describes the data it was designed and evaluated on. Data cards for every dataset are in `datacards/`.

Model details

Field	Value
Name	GreenShift scheduler, policy **P4** (multi-job greedy with an earliest-deadline-first feasibility check). P0 (cron), P2 (best window) and P3 (interruptible) ship alongside for comparison
Developers	Vismay HS (PES1UG24AM325), Sharvesh K (PES1UG24AM260), Ashwin C H (PES1UG24AM900). UE24AM343AB3 mini-project, Track G
Type	Deterministic combinatorial heuristic on a 30-minute UTC grid; no machine learning
Inputs	Each job's length (30-min slots), energy (kWh) and splittable flag; the night's window (release 22:00, deadline 07:00 UK); capacity *k* (parallel slots); the NESO national carbon-intensity **forecast**
Output	Each job's start and end time, the forecast intensity of its window and a per-job reason, saved to `results/plans/plan_<date>.json`
Code	`src/carbonsched/scheduler/policies.py`; live planner `scripts/run_scheduler.py`
Licence of inputs	Carbon Intensity API, NESO (CC BY 4.0)
Version	Repository `main`; the commit each output was built from is recorded in `results/report_summary.json`

How P4 decides. Jobs are taken largest energy first. Each takes the contiguous block with the lowest mean forecast intensity that keeps capacity ≤ *k* (splitting a job is P3's rule, not P4's). A move is accepted only if an EDF construction can still place every remaining job before 07:00. If the final plan's forecast emissions would exceed cron's, cron is kept.

Intended use

- **Primary use:** choosing when flexible nightly batch jobs (ETL, BI refreshes, reports) run inside a fixed overnight window on the GB grid, to lower their operational CO_2 without missing a hard deadline.
- **Intended users:** data and platform teams running scheduled Spark/YARN batch work; students and researchers studying temporal load shifting.
- **Out of scope:** latency-sensitive or interactive work; jobs whose deadline is earlier than their own length allows; regions outside GB (no forecast integration); claims of *marginal* avoided emissions; spatial shifting between regions; any safety-critical scheduling.

Factors

- **Season.** Savings depend on how much intensity falls overnight: largest in summer, smallest in winter.
- **Forecast quality.** The real forecast's error (MAE 7.6% of the mean) is the main factor limiting savings.
- **Capacity *k*.** With more parallel slots cron crowds work into the dirty early hours, so shifting gains more.
- **Job mix.** Lengths relative to the 9-hour window, and unequal energies (P4's largest-first ordering only matters then).
- **Clock changes.** Windows are 8 or 10 hours on clock-change nights; handled and tested.

Metrics

- **% CO_2 saved** = 1 − Σ policy grams / Σ cron grams, pooled over jobs and nights, scored with the **actual** intensity of the slots each job really occupied (never the forecast).
- **Uncertainty:** 95% percentile-bootstrap interval over nights (2,000 resamples).
- **Added delay:** each job's completion minus its own cron completion; median and p95.
- **Deadline misses** and **capacity violations:** must be 0; checked by an independent auditor.
- **Forecast regret:** share of the perfect-forecast (oracle) saving the real forecast achieves.

Evaluation data

Backtest on every night of 2024-2025 (729 nightly windows used; 10 night-scenarios with source outages skipped and listed in `backtest_meta.json`). Decisions use the archived NESO national forecast; scoring uses the national actual. Matrix: 4 policies × 3 forecast modes (real, noisy with 10 seeds, oracle) × $k \in \{1, 2\}$ × 5 jobs, plus a replication of Wiesner et al. Scenario I: 437,448 rows in total.

Training data

None: the scheduler has no learned parameters. It was designed against the 2024-2025 GB Carbon Intensity API history (data card). Its accounting was cross-checked against NESO's historic generation mix (r = 0.993, data card). The jobs are derived from TPC-DS (data card).

Quantitative analyses

The backtest uses modelled job lengths (30, 60, 60, 120 and 60 min) with equal energy per job, so results are reported as percentages.

P4, k = 1	% CO_2 saved [95% CI]	Median / p95 delay	Deadline misses
Real NESO forecast	**5.09** [4.53, 5.72]	2.5 h / 7 h	0
Noisy forecast (paper's model)	6.73 [6.14, 7.32]	2 h / 6.5 h	0
Perfect forecast (ceiling)	7.68 [7.10, 8.26]	1.5 h / 6.5 h	0

- With k = 2 parallel slots: **9.87%** [8.99, 10.76] (real forecast).
- By season (real forecast): winter 3.42%, spring 3.98%, summer **8.41%**, autumn 4.65%. Weekday nights 5.12% and weekend nights 5.03%.
- Per night: median 2.44%. **26.5% of nights emit more than cron** with the real forecast (0% with the oracle).
- The real forecast achieves 66% of the oracle saving (75% at k = 2).
- Replication (one 30-min job, ±2 h window, noisy forecast): 4.45% [4.01, 4.89] against the paper's 4.3%.
- The independent audit over all 437,448 rows found 0 deadline misses, 0 capacity violations and energy conserved.

All values: `report/results_tables.md`, recomputed from the raw rows in `notebooks/results.ipynb`.

Ethical considerations

- **Greenwashing risk.** Forecast-scored or oracle numbers would overstate the benefit. The headline is the real forecast scored on actual intensity, with an interval, and the losing nights are reported.
- **Average, not marginal, intensity.** Savings are attributional (location-based), as in the literature we compare with. The generator that actually responds to the extra load may differ.
- **Who bears the delay.** Downstream users of a moved job see its output later. Every job has a hard 07:00 deadline, delays are reported, and any job can be marked non-movable.
- **Transparency.** Every plan carries a per-job reason, and the website re-derives the plan in TypeScript and checks it against Python.
- **Rebound.** Cheaper-feeling compute can invite more work that cancels the saving.
- **No personal data** is used anywhere: grid aggregates and synthetic TPC-DS data only.

Caveats and recommendations

- Treat the numbers as **estimates for this job mix** (modelled lengths, equal energy). Measuring each job with the RAPL protocol would add absolute grams.
- RAPL covers the CPU package only (no DRAM, disk, network or power-supply losses), so measured grams would be a lower bound. Percentages are unaffected.
- Laptop-to-fleet figures use a stated SCALE and PUE = 1.0 and must be labelled *projection*.
- The live planner needs the public Carbon Intensity API. Offline, it falls back to a stored forecast and labels the plan as such. It plans only; it never starts Spark implicitly.
- Re-evaluate yearly: as the GB grid decarbonises, the overnight swing and the benefit of shifting change.